【马哥python说】Python入门私教课 | 第7讲–Python面向对象

视频讲解： B 站 · 第 7 讲 | **公众号：老男孩的平凡之路**

本课件基于 MIT License 开源，欢迎转载、分享，请保留作者与出处（详见仓库根目录 LICENSE）。

面向对象是一种编程思想，它把一类有共同特征的事物抽象成一个类（class），把其共有属性和共有方法封装到类里面。

一、类的定义和调用

类的定义有三种方式:

> 不带()：class Animal:
> 带()：class Animal():
> 带(object)：class Animal(object):

In [1]:
```python
# 定义一个动物的类
class Animal():
    def __init__(self, arg_name, arg_age):  # 构造方法，在类实例化时会自动调用
        self.name = arg_name  # 姓名
        self.age = arg_age  # 年龄

    # 定义一个类方法
    def eat(self):  # 吃饭
        print(self.name + "正在吃饭")

    # 定义一个类方法
    def sleep(self):  # 睡觉
        print(self.name + "正在睡觉")
```

In [2]:
```python
# 实例化一个对象
xiaobai = Animal(arg_name='小白', arg_age=3)
```

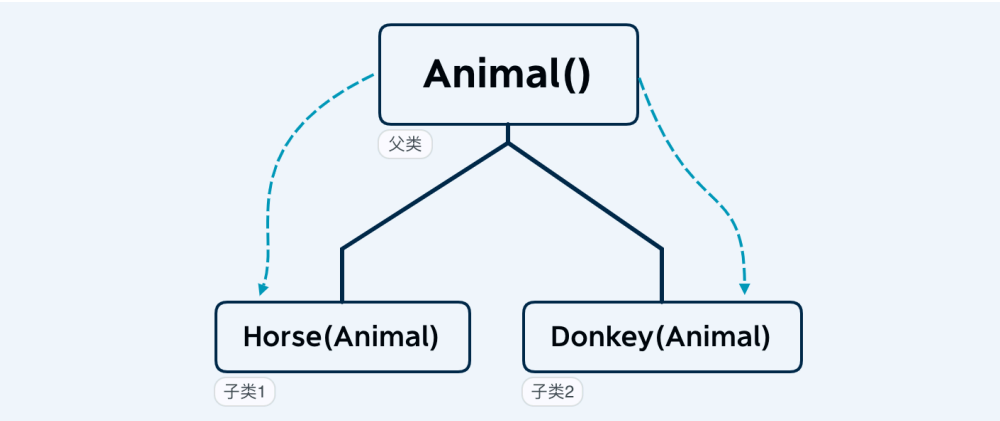
In [3]:
```python
# 调用类的属性
print('它的年龄是{}岁'.format(xiaobai.age))
```

它的年龄是3岁

In [4]:
```
# 调用类方法
xiaobai.sleep()
```

小白正在睡觉

二、类的继承



2.1 单继承1：Horse类继承自Animal类

In [33]:
```
# 定义一个马的类，继承自Animal类
class Horse(Animal):
    def __init__(self, arg_name, arg_age, arg_max_distance):  # 构造方法，在类实例化时会自动调用
        Animal.__init__(self, arg_name, arg_age)  # 调用父类的构造函数
        self.max_distance = arg_max_distance  # 最大奔跑距离

    def run(self):
        print(self.name + '正在奔跑')

    def sleep(self):  # 方法重写
        print(self.name + "正在马棚里睡觉")
```

In [34]:
```
horse1 = Horse(arg_name='白龙马', arg_age=5, arg_max_distance=20)
```

In [35]:
```
horse1.name  # 调用继承自父类的属性
```

Out[35]: '白龙马'

In [8]:
```
horse1.max_distance  # 调用子类的属性
```

Out[8]: 20

In [9]:
```
horse1.run()  # 调用子类的方法
```

白龙马正在奔跑

In [10]:
```
horse1.sleep()  # 调用子类的重写方法
```

白龙马正在马棚里睡觉

2.2 单继承2：Donkey类继承Animal类

In [11]:
```
# 定义一个驴的类，继承自Animal类
class Donkey(Animal):
    def __init__(self, arg_name, arg_age, arg_work_hours):  # 构造方法，在类实例化时会自动调用
        Animal.__init__(self, arg_name, arg_age)  # 调用父类的构造函数
        self.work_hours = arg_work_hours  # 工作时长

    def work(self):
        print(self.name + '正在拉磨')
```

In [12]:
```
donkey1 = Donkey(arg_name='小毛驴', arg_age='3', arg_work_hours=5)
```

In [13]:
```
donkey1.name  # 调用继承自父类的属性
```

Out[13]: '小毛驴'

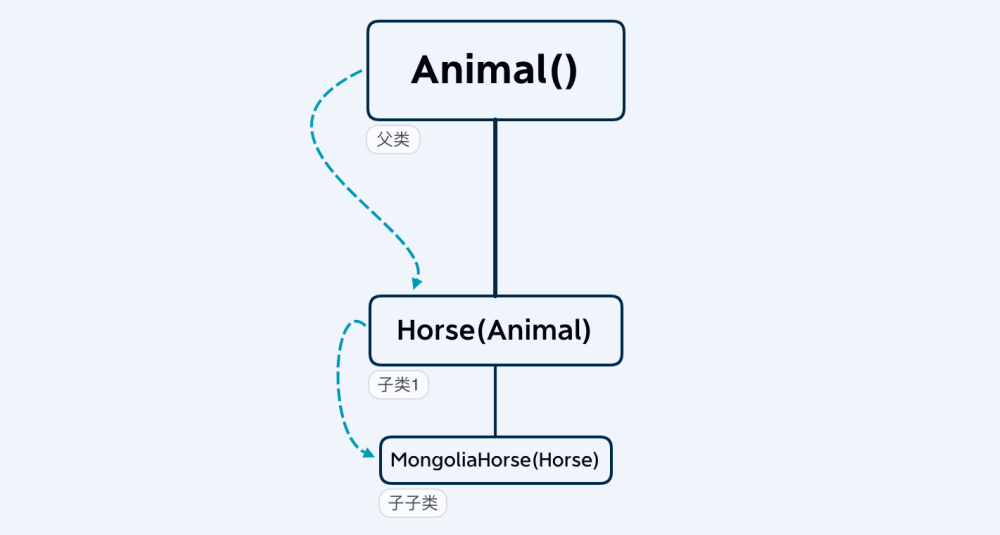
In [14]:
```
donkey1.work_hours  # 调用子类的属性
```

Out[14]: 5

In [15]:
```
donkey1.eat()  # 调用继承自父类的方法
```

小毛驴正在吃饭

2.3 多层继承



In [16]:
```
# 定义一个蒙古马的类，继承自Horse类
class MongoliaHorse(Horse):
    def race(self):
        print(self.name+'正在比赛')
```

In [17]:
```
mh1 = MongoliaHorse(arg_name='巴特尔', arg_age=7, arg_max_distance=35)
```

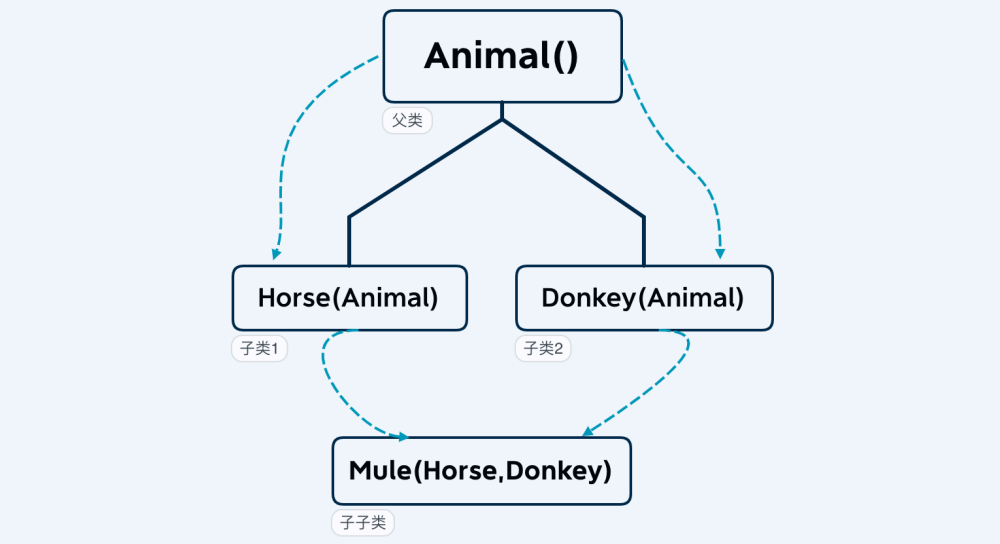
In [18]:
```
mh1.name
```

Out[18]: '巴特尔'

In [19]:
```
mh1.race()
```

巴特尔正在比赛

2.4 多重继承



In [20]:
```
class Mule(Horse, Donkey):
    def play(self):
        print(self.name+'正在玩耍')
```

In [21]:
```
mule1 = Mule(arg_name='小骡子', arg_age=4, arg_max_distance=20)
```

In [22]:
```
mule1.run()  # 继承自Horse类的方法
```

小骡子正在奔跑

In [23]:
```
mule1.work()  # 继承自Donkey类的方法
```

小骡子正在拉磨

In [24]:
```
mule1.play()  # 本类的方法
```

小骡子正在玩耍

三、类的私有属性

In [25]:
```
class Movie():
    def __init__(self, arg_name, arg_year):
        self.name = arg_name  # 电影名称
        self.__year = arg_year  # 上映年份

    def __get_info(self):  # 私有方法
        print('电影<{}>是{}年上映的'.format(self.name, self.__year))
```

In [26]:
```
movie1 = Movie(arg_name='大话西游', arg_year='1995')
```

In [27]:
```
movie1.__year  # 私有属性不可访问，所以报错了
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-27-f3753521a96e> in <module>
----> 1 movie1.__year  # 私有属性不可访问，所以报错了

AttributeError: 'Movie' object has no attribute '__year'
```

In [28]:
```
movie1.__get_info()  # 私有方法不可访问，所以报错了
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-28-e94126a07f58> in <module>
----> 1 movie1.__get_info()  # 私有方法不可访问，所以报错了

AttributeError: 'Movie' object has no attribute '__get_info'
```

In [29]:
```
movie1.name
```

Out[29]:
```
'大话西游'
```